

Bases de données managées

RDS : le même PostgreSQL, moins trois métiers

CL-AWS1 — Jour 5 — La base StockLine passe dans le cloud

Objectifs de la journée

À la fin de la journée, vous saurez :

- Expliquer **ce que « managé » change** : qui patche, qui sauvegarde, qui relance — et ce qui reste à votre charge.
- Créer une instance **RDS PostgreSQL** en CLI : moteur, classe, stockage, subnet group, security group.
- Choisir entre **Multi-AZ** (disponibilité) et **read replicas** (lectures) selon un besoin réel.
- Utiliser **sauvegardes automatiques, point-in-time recovery et snapshots** — et savoir ce qu'une restauration implique.
- Brancher **StockLine sur RDS** avec une seule variable : **DATABASE_URL**.
- Situer **Aurora** dans le paysage (survol).

Fil conducteur : au TP1 vous étiez le DBA de votre VM. Ce soir, vous aurez délégué le pire du métier.

Plan de la journée

1. **Pourquoi une base managée** — le vrai coût d'un PostgreSQL sur EC2.
2. **Anatomie d'une instance RDS** — moteur, classe, stockage, réseau.
3. **Multi-AZ vs read replicas** — deux réponses à deux problèmes différents.
4. **Sauvegardes et restauration** — snapshots, PITR, et leurs pièges.
5. **Démo : StockLine sur RDS** — **DATABASE_URL** fait tout le travail.
6. **Aurora en survol** — et récap, quiz, teardown.

Rythme : théorie ↔ pratique toutes les ~45 min, 2 exercices, quiz en fin de journée.

1. Pourquoi une base managée

Ce que vous faisiez au TP1 (et que vous avez déjà oublié)

Au TP1, StockLine tournait avec sa base **sur la même VM** :

- vous avez installé le moteur (paquets, dépendances) ;
- vous avez écrit un **script de sauvegarde** et son timer systemd ;
- vous étiez le seul recours si le processus mourait à 3 h du matin ;
- les mises à jour de sécurité du moteur ? Personne ne les faisait.

Et encore : c'était **SQLite**, le cas facile. Un vrai PostgreSQL de production ajoute : le tuning mémoire, la gestion des connexions, la réplication, les montées de version majeures...

Question à la salle : combien d'heures par mois pour bien faire tout ça ? C'est ce temps-là qu'on achète aujourd'hui.

PostgreSQL sur EC2 : tout est à vous

Rien ne vous interdit d'installer PostgreSQL sur une EC2 (c'est même parfois justifié : version exotique, extensions non supportées, contrôle total). Mais alors **tout** est à vous :

Tâche	PostgreSQL sur EC2
Installation, configuration du moteur	Vous
Patching OS et moteur	Vous
Sauvegardes + tests de restauration	Vous
Bascule si l'AZ tombe	Vous (et bon courage)
Réplication, montée de version	Vous
Supervision du moteur	Vous

Un DBA à temps partiel, caché dans votre fiche de poste d'admin cloud.

RDS : la responsabilité partagée, un cran plus loin

RDS (Relational Database Service) : AWS fait tourner le moteur pour vous. Vous n'avez **plus accès SSH** à la machine — et c'est le contrat :

Tâche	RDS
Matériel, OS, installation moteur	AWS
Patching (fenêtre de maintenance)	AWS
Sauvegardes automatiques + PITR	AWS
Bascule Multi-AZ automatique	AWS
Schéma, requêtes, index	Vous
Utilisateurs SQL, droits, mots de passe	Vous
Choix de classe, réglages, coûts	Vous

Rappel J1 : la responsabilité partagée n'est pas un slogan — c'est une **liste de tâches** qui change de colonne.

Ce que RDS ne fera jamais pour vous

Pour éviter le malentendu classique, RDS **ne gère pas** :

- **vos données** : une table mal conçue reste mal conçue ;
- **vos requêtes** : le **SELECT *** sans index sera lent, managé ou pas ;
- **vos erreurs** : un **DELETE** sans **WHERE** est répliqué fidèlement partout (on y revient en section 4...) ;
- **votre sécurité applicative** : mot de passe faible, injection SQL — toujours votre problème ;
- **votre facture** : une classe surdimensionnée facture 24 h/24.

« Managé » = AWS gère la **machinerie**, pas votre **métier**.

2. Anatomie d'une instance RDS

Les moteurs disponibles

RDS fait tourner des moteurs **standards** — vos compétences SQL restent valables :

- **PostgreSQL** ← notre choix (StockLine tourne dessus depuis CL-PYTHON) ;
- **MySQL** et **MariaDB** — les classiques du web ;
- **Oracle** et **SQL Server** — les licences d'entreprise (coût !) ;
- **Aurora** — le moteur maison d'AWS, compatible PostgreSQL/MySQL (section 6).

Point important : c'est un **vrai** PostgreSQL. **psql**, SQLAlchemy, pgAdmin, vos dumps : tout fonctionne à l'identique. Seul l'accès à l'OS disparaît.

Choisir un moteur = choisir pour des années (les migrations de moteur sont des projets entiers). En sortie de formation : PostgreSQL sauf contrainte client.

Classes d'instance et 💰 free-tier

Comme EC2, RDS se décline en **classes** (**db.** + famille + taille) :

- **db.t3.micro**, **db.t4g.micro** — burstables, dev/test, **free-tier** ;
- **db.m6i.large**... — usage général, production ;
- **db.r6i.large**... — optimisées mémoire (les bases aiment la RAM).

💰 Encadré coût — RDS free-tier (nos TP)

- **750 h/mois** de **db.t3.micro** (une instance en continu) ;
- **20 Go** de stockage SSD + 20 Go de sauvegardes ;
- **Single-AZ uniquement** : activer Multi-AZ = facturé (×2) ;
- une 2^e instance simultanée (ex. restauration gardée) = dépassement. Hors free-tier, **db.t3.micro** ≈ 0,018 /h ; **db.m5.large** ≈ 0,17/h — **choisir la classe est un acte de gestion.**

Le stockage

Le stockage RDS est de l'EBS géré, dimensionné à la création (extensible ensuite, jamais réductible) :

- **gp3** (SSD usage général) : notre choix — 3 000 IOPS de base, tarif ~0,088 \$/Go/mois hors free-tier ;

- **io1/io2** (IOPS provisionnées) : bases exigeantes, cher ;
- **magnetic** : héritage, à éviter.

Deux réglages à connaître :

- `--allocated-storage 20` : les 20 Go du free-tier ;
- **storage autoscaling** : RDS peut agrandir tout seul quand le disque se remplit — pratique en prod, à **plafonner** (sinon la facture s'autoscale aussi).

Réflexe : un disque de base plein = base en panne. La métrique `FreeStorageSpace` sera dans nos alarmes du J8.

Le DB subnet group : dire à RDS où habiter

RDS ne vous demande pas UN sous-réseau mais un **groupe de sous-réseaux** :

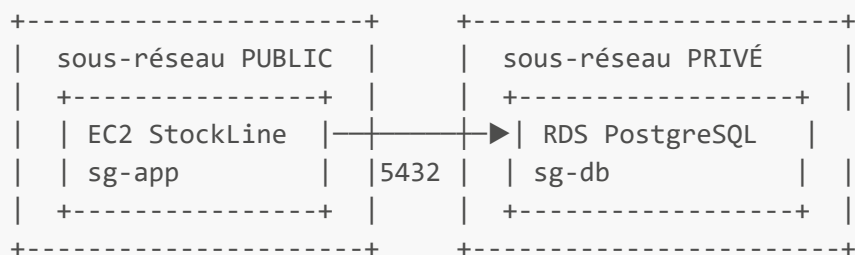
- couvrant **au moins 2 AZ** (obligatoire, même en single-AZ) ;
- des sous-réseaux **privés** : une base n'a rien à faire sur Internet.

```
aws rds create-db-subnet-group \
  --db-subnet-group-name ${PREFIX}-stockline-subnets \
  --db-subnet-group-description "Sous-reseaux prives StockLine" \
  --subnet-ids $SUBNET_PRIVATE_A $SUBNET_PRIVATE_B
```

Pourquoi 2 AZ dès maintenant ? Pour que la bascule **Multi-AZ** (section 3) soit possible sans reconstruire. Le VPC du J3 avait déjà tout prévu.

Le security group de la base : la chaîne complète

Internet —X— (rien n'entre)
VPC 10.0.0.0/16



sg-db : entrée 5432/tcp UNIQUEMENT depuis sg-app
(référence de SG à SG — jamais 0.0.0.0/0)

L'endpoint : un nom, jamais une IP

Une fois la base `available`, RDS fournit un **endpoint** :

```
abc-stockline-db.cx3k9q2p7r1m.eu-west-3.rds.amazonaws.com:5432
```

- C'est un **nom DNS** : l'IP derrière peut changer (bascule Multi-AZ, maintenance) — le nom, lui, reste.
- D'où la règle absolue : **jamais d'IP de base codée en dur**. Ni dans le code, ni dans la config.
- `--no-publicly-accessible` : pas d'IP publique du tout. Le nom ne se résout qu'en IP privée, depuis le VPC.

```
aws rds describe-db-instances \
  --db-instance-identifier ${PREFIX}-stockline-db \
  --query 'DBInstances[0].Endpoint.Address' --output text
```

Paramètres et maintenance

Plus d'accès à `postgresql.conf` ? Les équivalents managés :

- **Parameter group** : les réglages du moteur (mémoire partagée, logs des requêtes lentes...). On ne modifie jamais le groupe par défaut : on en **crée un** et on l'attache. Certains paramètres exigent un redémarrage.
- **Option group** : fonctionnalités additionnelles (surtout Oracle/SQL Server).
- **Fenêtre de maintenance** (`PreferredMaintenanceWindow`) : le créneau hebdomadaire où AWS a le droit de patcher — à placer la nuit, un jour creux. Les mises à jour **mineures** peuvent être automatiques ; les **majeures** attendent toujours votre feu vert.
- **Fenêtre de sauvegarde** (`PreferredBackupWindow`) : idem pour le backup quotidien.

Réflexe pro : ces deux fenêtres se **choisissent** (sinon AWS tire au sort — peut-être mardi 14 h).

3. Multi-AZ vs read replicas

Deux problèmes, deux mécanismes — ne les confondez jamais

Le problème n° 1 : la panne

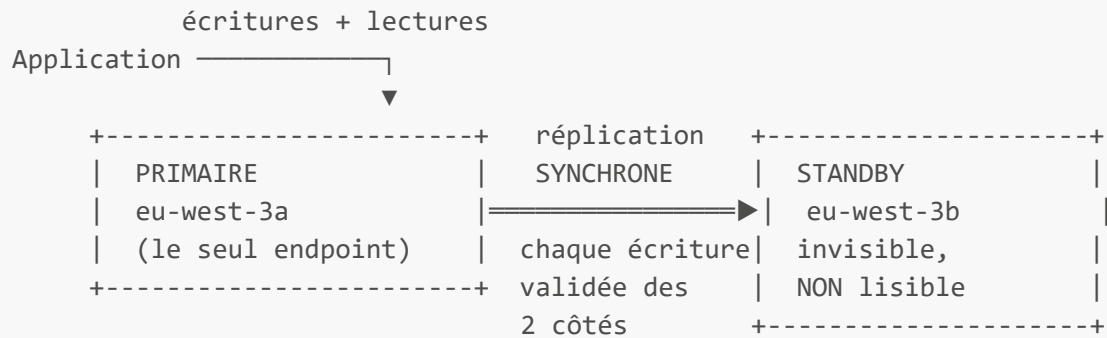
Votre base single-AZ tourne dans `eu-west-3a`. Que se passe-t-il si :

- l'hôte physique meurt ? → RDS relance ailleurs **dans la même AZ** : quelques minutes d'arrêt, données préservées (EBS).
- **l'AZ entière tombe** (incendie, coupure — c'est arrivé) ? → votre base est **indisponible** tant que l'AZ l'est. Restauration possible dans une autre AZ **depuis la sauvegarde** : des heures, et perte des dernières transactions.

Pour une application de vente, chaque minute d'arrêt de la base = zéro chiffre d'affaires.

La réponse RDS : **Multi-AZ** — une copie de secours, ailleurs, toujours à jour.

Multi-AZ : le schéma



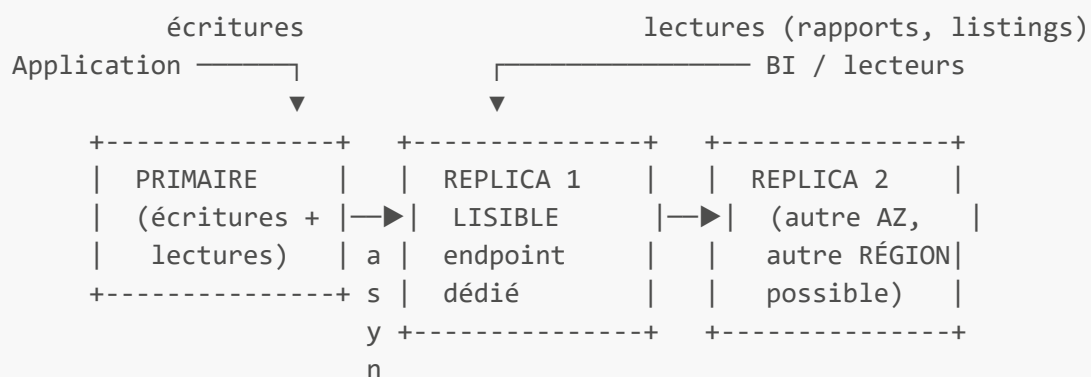
PANNE du primaire : bascule AUTOMATIQUE en 1-2 min
 le nom DNS (endpoint) pointe alors vers l'ex-standby
 l'application ne change RIEN (d'où : jamais d'IP en dur)

Multi-AZ : ce qu'il faut savoir (et réciter)

- Réplication **synchrone** : une écriture n'est validée que lorsque les **deux** copies l'ont — perte de données quasi nulle (RPO $\approx$ 0).
- Bascule **automatique** en 1 à 2 minutes (RTO court) : panne, mais aussi maintenance — AWS patche le standby, bascule, patche l'autre.
- Le standby est **invisible** : on ne peut **pas** lire dessus. Zéro gain de performance. C'est une assurance, pas un turbo.
- Activation en une commande (`--multi-az`), bascule de test possible (`reboot with failover`).
- 💰 **prix x2** environ (deux instances, deux stockages) — hors free-tier.

Phrase d'examen : « *Multi-AZ = haute disponibilité, pas performance.* »

Read replicas : le schéma



Read replicas : ce qu'il faut savoir

- Réplication **asynchrone** : le réplica suit, avec un léger retard (*replica lag*) — acceptable pour un rapport, pas pour un solde bancaire.
- Chaque réplica a **son propre endpoint** : c'est l'**application** qui décide d'envoyer ses lectures dessus (ce n'est pas transparent).
- Usage type : télécharger les **SELECT** massifs (BI, exports, pages catalogues) — le primaire garde les écritures.
- **Pas de bascule automatique** : en cas de panne du primaire, on peut **promouvoir** un réplica... à la main, et ça casse la réplication.
- Possible **cross-region** (lectures locales à l'étranger, plan de secours géographique).
- Multi-AZ et réplicas **se combinent** : un primaire Multi-AZ + des réplicas de lecture.

Phrase d'examen : « *read replica = performance en lecture, pas haute dispo.* »

Le tableau à connaître par cœur

	Multi-AZ	Read replica
Problème résolu	Panne (disponibilité)	Charge en lecture (performance)
Réplication	Synchrone	Asynchrone (lag)
Copie lisible ?	✗ Non, invisible	✓ Oui, endpoint dédié
Bascule	Automatique (1-2 min)	Manuelle (promotion)
Perte de données	≈ 0 (RPO ≈ 0)	Dernières secondes possibles
Portée	Même région (2 AZ)	Même région ou cross-region
Protège d'un DELETE raté ?	✗ (répliqué aussi !)	✗ (répliqué aussi !)
Coût	×2	+1 instance par réplica

La dernière ligne pique : contre l'erreur humaine, **ni l'un ni l'autre**. Il faut la section 4.



Exercice 5-1 — Multi-AZ ou replica ? Quatre clients, quatre verdicts

Fichier : [exercices/06-cl-aws1/exercice-5-1-multi-az-ou-replica.md](#) — **30 min, en binômes**

- Quatre scénarios clients (billetterie, média, BI du lundi, outil interne) : pour chacun, un **verdict** (Multi-AZ, replica, les deux, rien), une justification, l'impact de coût.
- Puis cinq affirmations d'un junior à valider ou corriger.

La grille : « **panne ou charge ? lecture ou écriture ?** » — deux questions, le verdict tombe seul.

Sur papier : zéro ressource AWS, zéro coût. Correction en binômes croisés puis mise en commun.

4. Sauvegardes et restauration

La seule protection contre **DELETE FROM produits;**

Sauvegardes automatiques et point-in-time recovery

Avec `--backup-retention-period 7` (0 = désactivé, max 35 jours), RDS fait chaque jour, dans la fenêtre choisie :

- un **snapshot complet** du stockage ;
- et il archive **en continu les journaux de transactions**.

La combinaison des deux = **PITR (Point-In-Time Recovery)** : restaurer l'état de la base à **n'importe quelle seconde** de la fenêtre de rétention.

snapshot 03h00 — transactions archivées en continu —> maintenant
 ↳ restauration possible à CHAQUE instant de la flèche

C'est LA parade à l'erreur humaine : le **DELETE** de vendredi 18 h 30 ? On restaure à 18 h 29.

Snapshots manuels : avant chaque opération risquée

```
# Avant une migration, un gros script, une suppression :
aws rds create-db-snapshot \
  --db-instance-identifiant ${PREFIX}-stockline-db \
  --db-snapshot-identifiant ${PREFIX}-stockline-avant-migration
```

Différence de **cycle de vie** — question d'examen et de facture :

	Automatique	Manuel
Créé par	RDS, chaque jour	Vous, à la demande
Expire	Fin de rétention (7 j)	Jamais
Si l'instance est supprimée	Supprimé avec elle	Survit (et facture !)

Les snapshots manuels oubliés sont un grand classique du gaspillage (rendez-vous à l'exercice 8-1).

Restaurer = créer une NOUVELLE instance

La surprise de tous les débutants : la restauration **ne modifie jamais** la base d'origine. Elle crée une **instance neuve** (nouvel identifiant, **nouvel endpoint**) :

```
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifiant ${PREFIX}-stockline-db \
  --target-db-instance-identifiant ${PREFIX}-stockline-restauree \
  --restore-time 2026-06-26T16:29:00Z \
  --db-subnet-group-name ${PREFIX}-stockline-subnets \
  --vpc-security-group-ids $SG_DB --no-publicly-accessible
```

Trois conséquences pratiques :

- il faut **rebrancher l'application** (nouvel endpoint dans `DATABASE_URL`) ;
- les paramètres réseau (subnet group, SG) doivent être **redonnés** ;
- les horodatages AWS sont en **UTC** (18 h 30 à Paris l'été = 16 h 30 Z).



Exercice 5-2 — Snapshots et point-in-time : sauver la base StockLine

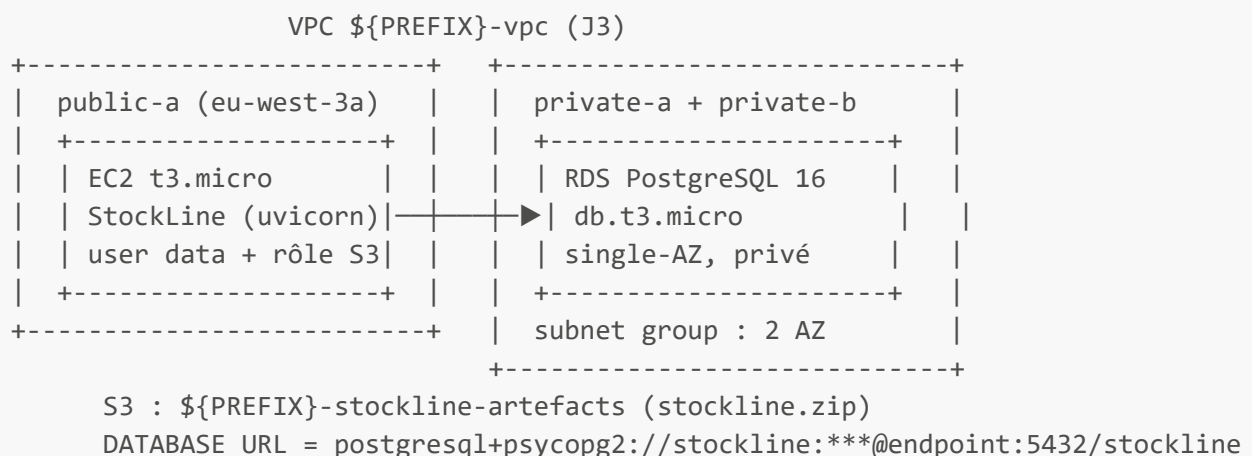
Fichier : `exercices/06-cl-aws1/exercice-5-2-sauvegards-et-restauration.md` — **45 min, seul(e)**

- Partie 1 (CLI, sur votre sandbox) : créer, lister (avec `--query` !), supprimer un snapshot manuel.
- Partie 2 (sur papier) : l'incident du lundi matin — le **DELETE** de vendredi soir. Fenêtres, UTC, commande de restauration, et le vrai problème : réconcilier les données écrites depuis.
- Partie 3 : nettoyage prouvé.

⚠ On n'exécute **pas** la restauration (2^e instance = hors free-tier) — on l'écrit, on la comprend.

5. Démo — StockLine sur RDS

L'architecture de la démo



Démo 5-1 — Une base PostgreSQL managée pour StockLine

Fichier : `demos/06-cl-aws1/demo-5-1-rds-stockline.md` — 60 min

Au programme, tout en CLI :

1. le **subnet group** sur les 2 sous-réseaux privés du J3 ;
2. le **SG de base** (5432 depuis le SG applicatif — jamais d'IP) ;
3. `create-db-instance` : PostgreSQL 16, db.t3.micro, 20 Go gp3, rétention 7 j, **privé** ;
4. une EC2 qui installe StockLine toute seule (`user-data-stockline.sh`) et `/sante` qui répond `"base_de_donnees":"ok"` ;
5. la preuve du managé : **snapshot en 1 commande, reboot sans nous**.

Suivez avec `code/06-cl-aws1/user-data-stockline.sh` sous les yeux : chaque ligne est commentée.

Ce que la démo vient de prouver : la config par l'environnement

Le code de StockLine n'a **pas changé d'une ligne** depuis CL-PYTHON :

```
# app/db.py — écrit au bloc 02, jamais retouché depuis :  
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./stockline.db")  
engine = create_engine(DATABASE_URL, ...)
```

- TP1 : pas de variable → SQLite locale.
- Aujourd'hui : `DATABASE_URL=postgresql+psycopg2://...rds.amazonaws.com:5432/stockline` → RDS.
- Demain (J6) : la **même** variable dans un launch template → toutes les instances de l'ASG partagent la même base.

C'est le principe « **la configuration vit dans l'environnement, pas dans le code** » — posé en CL-PYTHON, il paie aujourd'hui. Le mot de passe, lui, attend Secrets Manager (CL-SECU).

6. Aurora en survol

Aurora : PostgreSQL, réinventé par AWS

Aurora = le moteur propriétaire d'AWS, **compatible** PostgreSQL ou MySQL (vos applications ne voient pas la différence) — mais l'architecture interne change tout :

- le **stockage est séparé du calcul** : un volume distribué, répliqué **6 fois sur 3 AZ**, auto-réparant, qui grandit tout seul ;
 - jusqu'à **15 réplicas** de lecture, avec bascule automatique en ~30 s (les réplicas SONT le standby : HA et lectures enfin réunis) ;
 - des variantes : **Aurora Serverless** (capacité qui suit la charge), **Global Database** (réplication multi-régions ~1 s).
-

Aurora vs RDS classique : quand choisir quoi

Critère	RDS PostgreSQL	Aurora PostgreSQL
Compatibilité	PostgreSQL natif	Compatible PostgreSQL
HA	Multi-AZ (1 standby)	6 copies / 3 AZ, bascule ~30 s
Réplicas	5, asynchrones	15, lag minime
Prix	💰 modéré, free-tier	💰💰 ~20 % + cher, pas de free-tier
Pour nous	✅ Formation, TP2	Survot (revu en CL-CERT)

À retenir pour l'examen : gros trafic de lecture + exigence de bascule rapide + budget → Aurora. Besoin standard, budget maîtrisé, version précise de PostgreSQL → RDS.

Pour le TP2 : **RDS PostgreSQL**, sans hésitation.

Récap — les points clés du jour

- **Managé** = AWS prend la machinerie (patch, backup, bascule), vous gardez le métier (schéma, requêtes, coûts).
- **db.t3.micro + 20 Go + single-AZ = free-tier** ; tout le reste se paie — la classe est un choix de gestion. 💰
- **Subnet group ≥ 2 AZ, sous-réseaux privés, SG référencé par SG** : la base n'est joignable que par l'application.
- **Multi-AZ = disponibilité** (synchrone, invisible, bascule auto) ≠ **replica = lectures** (asynchrone, lisible, promotion manuelle).
- Contre l'erreur humaine : **PITR et snapshots** — et restaurer crée une **nouvelle** instance.
- L'endpoint est un **nom DNS** ; l'application se branche par **DATABASE_URL**.

🔪 Teardown du soir — non négociable

Ce soir, il ne reste RIEN qui facture :

```
# 1. L'EC2 de démo :
aws ec2 terminate-instances --region eu-west-3 --instance-ids <INSTANCE_ID>

# 2. La base (sauf consigne "on la garde pour demain") :
aws rds delete-db-instance --region eu-west-3 \
  --db-instance-identifier ${PREFIX}-stockline-db \
  --final-db-snapshot-identifier ${PREFIX}-stockline-fin-j5

# 3. Le VPC et sa NAT Gateway 💰 :
./teardown-vpc.sh

# 4. La preuve :
aws rds describe-db-instances --region eu-west-3 --query
'DBInstances[].DBInstanceIdentifier'
```

```
aws ec2 describe-nat-gateways --region eu-west-3 \
  --filter "Name=state,Values=available" --query 'NatGateways[].NatGatewayId'
```

Quiz du jour (1/5)

Question 1 — Avec RDS, qui est responsable du patching du système d'exploitation qui porte la base, et qui est responsable de la qualité des index de la table `mouvements` ?

Question 2 — Votre collègue affirme : « on a activé le Multi-AZ, on peut donc envoyer les rapports BI sur la deuxième instance pour soulager la première ». Que lui répondez-vous ?

Quiz du jour (2/5)

Question 3 — Quel type de réplication utilise le Multi-AZ, et quel type utilise un read replica ? Quelle conséquence concrète cette différence a-t-elle sur les données lues depuis un réplica ?

Question 4 — Un stagiaire exécute `DELETE FROM produits;` sans `WHERE` sur la base de production, qui est en Multi-AZ avec un read replica. Quelles copies de la donnée sont touchées, et quel mécanisme permet de récupérer ?

Quiz du jour (3/5)

Question 5 — Pourquoi l'endpoint RDS est-il fourni sous forme de nom DNS plutôt que d'adresse IP, et quelle mauvaise pratique applicative cela interdit-il ?

Question 6 — Citez les trois conditions pour qu'une instance RDS reste dans le free-tier (classe, stockage, topologie).

Quiz du jour (4/5)

Question 7 — Pourquoi RDS exige-t-il un DB subnet group couvrant au moins deux AZ, même pour une instance single-AZ ?

Question 8 — Vous supprimez l'instance `${PREFIX}-stockline-db`. Que deviennent (a) ses snapshots automatiques, (b) ses snapshots manuels ? Quelle conséquence sur la facture ?

Quiz du jour (5/5)

Question 9 — Après une restauration point-in-time réussie, l'application affiche toujours les données corrompues. Quelle est l'explication la plus probable, et que faut-il changer ?

Question 10 — Citez deux différences majeures entre Aurora et RDS PostgreSQL classique, et la raison pour laquelle le TP2 utilisera RDS classique.

Réponses détaillées : guide formateur du bloc (jours 5-8).

À demain !

Jour 6 — Haute disponibilité

Votre base est solide... mais StockLine tourne encore sur **une seule** instance EC2. Demain : un load balancer, un Auto Scaling Group — et on **tuera une instance en direct** pour prouver que le service ne tombe plus.

✂ Avant de partir : teardown exécuté, compte vide, `rds-ids.env` conservé si consigne.